TOPIC 12

Task 1

```
import Foundation
protocol Borrowable {
    var id: Int { get }
    var title: String { get }
    var isAvailable: Bool { get set }
}
struct Book: Borrowable {
    let id: Int
    let title: String
    let author: String
    var isAvailable = true
}
struct Magazine: Borrowable {
    let id: Int
    let title: String
    let issue: Int
    var isAvailable = true
}
enum LibraryError: Error {
    case itemNotFound(id: Int)
    case alreadyBorrowed(title: String)
    case limitReached(max: Int)
}
final class Member {
    let name: String
    private(set) var borrowedIDs: [Int] = []
    static let limit = 2
```

```
    init(name: String) {
        self.name = name
    }
    func recordBorrow(_ id: Int) throws {
        if borrowedIDs.count >= Member.limit {
            throw LibraryError.limitReached(max: Member.limit)
        }
        borrowedIDs.append(id)
    }
    func recordReturn(_ id: Int) {
        borrowedIDs.removeAll { $0 == id }
    }
}
struct Catalog<Item: Borrowable> {
    var items: [Item] = []
    mutating func add(_ item: Item) {
        items.append(item)
    }
    mutating func checkout(id: Int, to member: Member) throws {
        // Find the item
        guard let index = items.firstIndex(where: { $0.id == id }) else {
            throw LibraryError.itemNotFound(id: id)
        }
        if !items[index].isAvailable {
            throw LibraryError.alreadyBorrowed(title: items[index].title)
        }
        try member.recordBorrow(id)
        items[index].isAvailable = false
        print("\(items[index].title) borrowed successfully")
```

```
    }
}
var catalog = Catalog<Book>()
catalog.add(Book(id: 1, title: "Swift Basics", author: "John"))
catalog.add(Book(id: 2, title: "iOS Development", author: "Sam"))
catalog.add(Book(id: 3, title: "Swift Programming", author: "Alex"))
let member = Member(name: "Anu")
do {
    try catalog.checkout(id: 1, to: member)
    do {
        try catalog.checkout(id: 1, to: member)
    } catch LibraryError.alreadyBorrowed(let title) {
        print("\(title) is already borrowed")
    }
    try catalog.checkout(id: 2, to: member)
    try catalog.checkout(id: 3, to: member)
} catch LibraryError.limitReached(let max) {
    print("Borrowing limit reached: \(max) items")
} catch {
    print("Error occurred")
}
print("Borrowed IDs:", member.borrowedIDs)
```

OUTPUT:

```
Swift Basics borrowed successfully
Swift Basics is already borrowed
iOS Development borrowed successfully
Borrowing limit reached: 2 items
Borrowed IDs: [1, 2]
```

Task 2:

```
import Foundation
protocol Entity {
    var id: Int { get }
}
enum RepositoryError: Error {
    case duplicateID
    case notFound
}
struct Student: Entity {
    let id: Int
    let name: String
    let marks: Int
}
struct Course: Entity {
    let id: Int
    let title: String
}
struct Repository<T: Entity> {
    var items: [Int: T] = [:]
    mutating func add(_ item: T) throws {
        if items[item.id] != nil {
            throw RepositoryError.duplicateID
        }

        items[item.id] = item
    }
```

```
    func find(_ id: Int) -> T? {
        return items[id]
    }
    mutating func remove(_ id: Int) throws {
        if items[id] == nil {
            throw RepositoryError.notFound
        }
        items.removeValue(forKey: id)
    }
    func all(where condition: (T) -> Bool) -> [T] {
        return items.values
            .filter { condition($0) }
            .sorted { $0.id < $1.id }
    }
}
var studentRepo = Repository<Student>()
do {
    try studentRepo.add(Student(id: 3, name: "Meena", marks: 90))
    try studentRepo.add(Student(id: 1, name: "Anu", marks: 80))
    try studentRepo.add(Student(id: 2, name: "Ravi", marks: 60))
    if let student = studentRepo.find(1) {
        print("Found:", student.name)
    }
    let passed = studentRepo.all { $0.marks >= 70 }
    for student in passed {
        print(student.id, student.name, student.marks)
    }
    try studentRepo.remove(2)
```

```
    print("Student removed")
} catch RepositoryError.duplicateID {
    print("ID already exists")
} catch RepositoryError.notFound {
    print("Student not found")
} catch {
    print("Some error occurred")
}
var courseRepo = Repository<Course>()
do {
    try courseRepo.add(Course(id: 101, title: "Swift"))
    try courseRepo.add(Course(id: 102, title: "iOS Development"))
    if let course = courseRepo.find(101) {
        print("Course:", course.title)
    }
} catch {
    print("Course error")
```

}OUTPUT:

```
Found: Anu
1 Anu 80
3 Meena 90
Student removed
Course: Swift
```

Task 3:

```
import Foundation
protocol Entity {
    var id: Int { get }
}
protocol Describable {
```

```
    var summary: String { get }
}
enum RepositoryError: Error {
    case duplicateID
    case notFound
}
struct Student: Entity, Describable {
    let id: Int
    let name: String
    let marks: Int
    var summary: String {
        return "ID: \(id), Name: \(name), Marks: \(marks)"
    }
}
struct Course: Entity, Describable {
    let id: Int
    let title: String

    var summary: String {
        return "ID: \(id), Course: \(title)"
    }
}
struct Repository<T: Entity> {
    var items: [Int: T] = [:]
    mutating func add(_ item: T) throws {
        if items[item.id] != nil {
            throw RepositoryError.duplicateID
        }
        items[item.id] = item
```

```
    }
    func find(_ id: Int) -> T? {
        return items[id]
    }
    mutating func remove(_ id: Int) throws {
        if items[id] == nil {
            throw RepositoryError.notFound
        }

        items.removeValue(forKey: id)
    }
    func all(where condition: (T) -> Bool) -> [T] {
        return items.values
            .filter { condition($0) }
            .sorted { $0.id < $1.id }
    }
}
extension Repository where T: Describable {
    func printReport() {
        let sortedItems = items.values.sorted { $0.id < $1.id }

        for item in sortedItems {
            print(item.summary)
        }
    }
}
var studentRepo = Repository<Student>()
do {
    try studentRepo.add(Student(id: 2, name: "Ravi", marks: 75))
```

```
    try studentRepo.add(Student(id: 1, name: "Anu", marks: 90))
    print("Student Report:")
    studentRepo.printReport()
  if let student = studentRepo.find(1) {
        print("Found Student:", student.name)
    }
} catch RepositoryError.duplicateID {
    print("Student ID already exists")
} catch {
    print("Student error")
}
var courseRepo = Repository<Course>()

do {
    try courseRepo.add(Course(id: 102, title: "UIKit"))
    try courseRepo.add(Course(id: 101, title: "Swift"))
    print("\nCourse Report:")
    courseRepo.printReport()
} catch {
    print("Course error")
}
struct Product: Entity {
    let id: Int
    let name: String
}
var productRepo = Repository<Product>()
do {
    try productRepo.add(Product(id: 1, name: "Laptop"))
    print("\nProduct added successfully")
```

```
} catch {
    print("Product error")
}
```

OUTPUT:

```
Student Report:
ID: 1, Name: Anu, Marks: 90
ID: 2, Name: Ravi, Marks: 75
Found Student: Anu

Course Report:
ID: 101, Course: Swift
ID: 102, Course: UIKit

Product added successfully
```